

Java Ticketing System

(Project overview, features, deep-dive concepts, testing plans, and links)

Project Overview

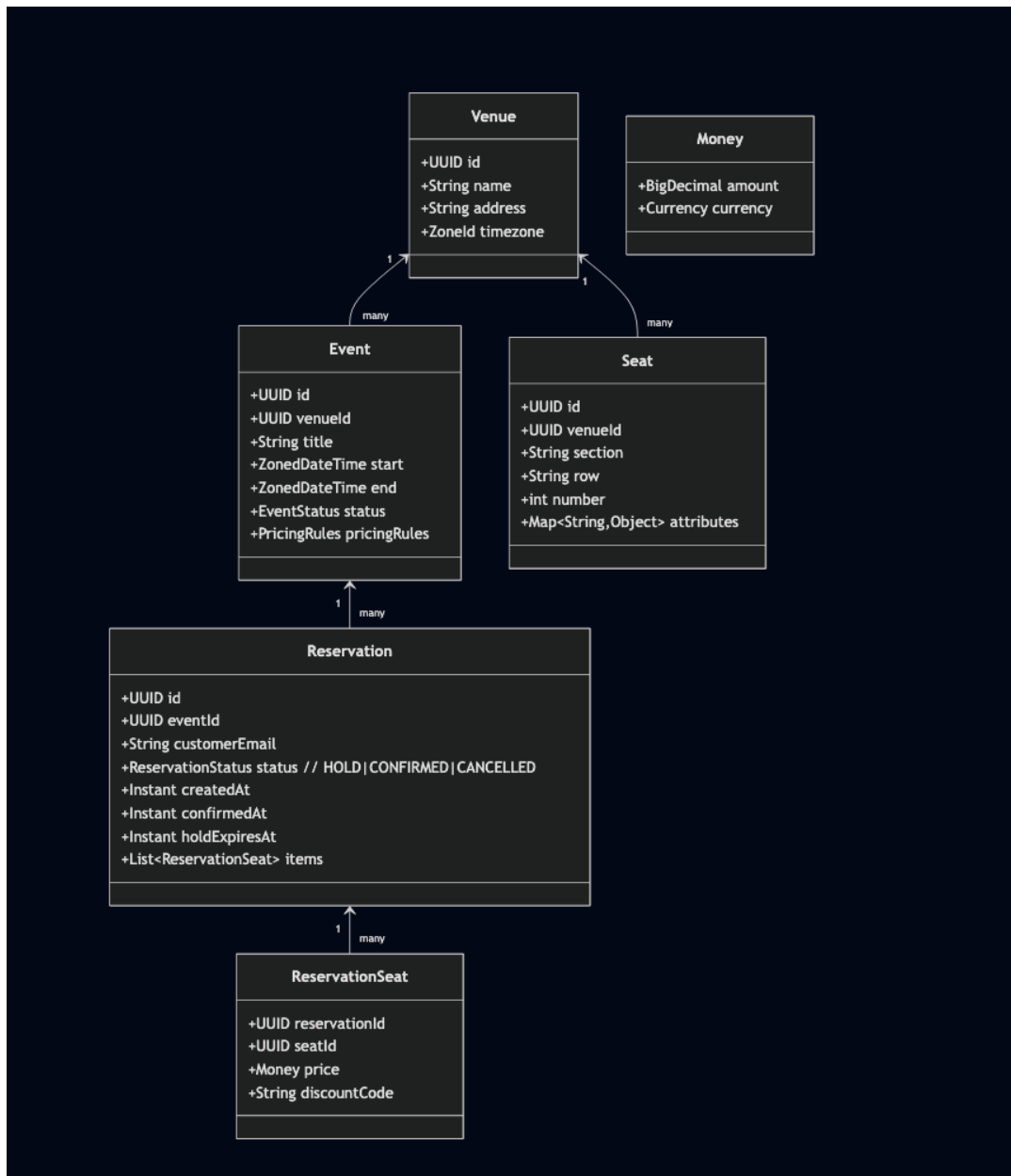
A ticketing system for venues and events. Users browse events, view seat availability, place **holds** (temporary reservations), **confirm** reservations, **cancel** when allowed, **export tickets**, and run basic **reports**. The system starts as a **pure Java CLI**, evolves to a **Spring Boot + PostgreSQL** REST API, and finally adds **security, observability, performance, and packaging**.

Primary Users

- **Customer:** browse, hold seats, confirm/cancel, download ticket.
- **Operator:** import seats/events, view summaries/reports, basic admin tasks.

Core Entities (Domain Model)

- **Venue** (id, name, address, timezone)
- **Event** (id, venueId, title, start/end, status, pricingRules)
- **Seat** (id, venueId, section, row, number, attributes)
- **Reservation** (id, eventId, customerEmail, status = HOLD | CONFIRMED | CANCELLED, createdAt, confirmedAt, holdExpiresAt)
- **ReservationSeat** (reservationId, seatId, price, discountCode?)



Domain model overview: Venue, Event, Seat, Reservation, Reservation Seat.

Initial Data Model (hardened in Phase 3)

- **venues** (id PK, name, timezone, address, created_at)
- **events** (id PK, venue_id FK, title, start_at, end_at, status, created_at, **UNIQUE(venue_id, title, start_at)**)
- **seats** (id PK, venue_id FK, section, row, number, attributes JSONB, **UNIQUE(venue_id, section, row, number)**)

- **reservations** (id PK, event_id FK, customer_email, status, hold_expires_at, created_at, confirmed_at)
- **reservation_seats** (reservation_id FK, seat_id FK, price, discount_code, UNIQUE(event_id, seat_id))

Phase 1 — Core Java CLI (Foundations)

Concepts You'll Learn (in depth)

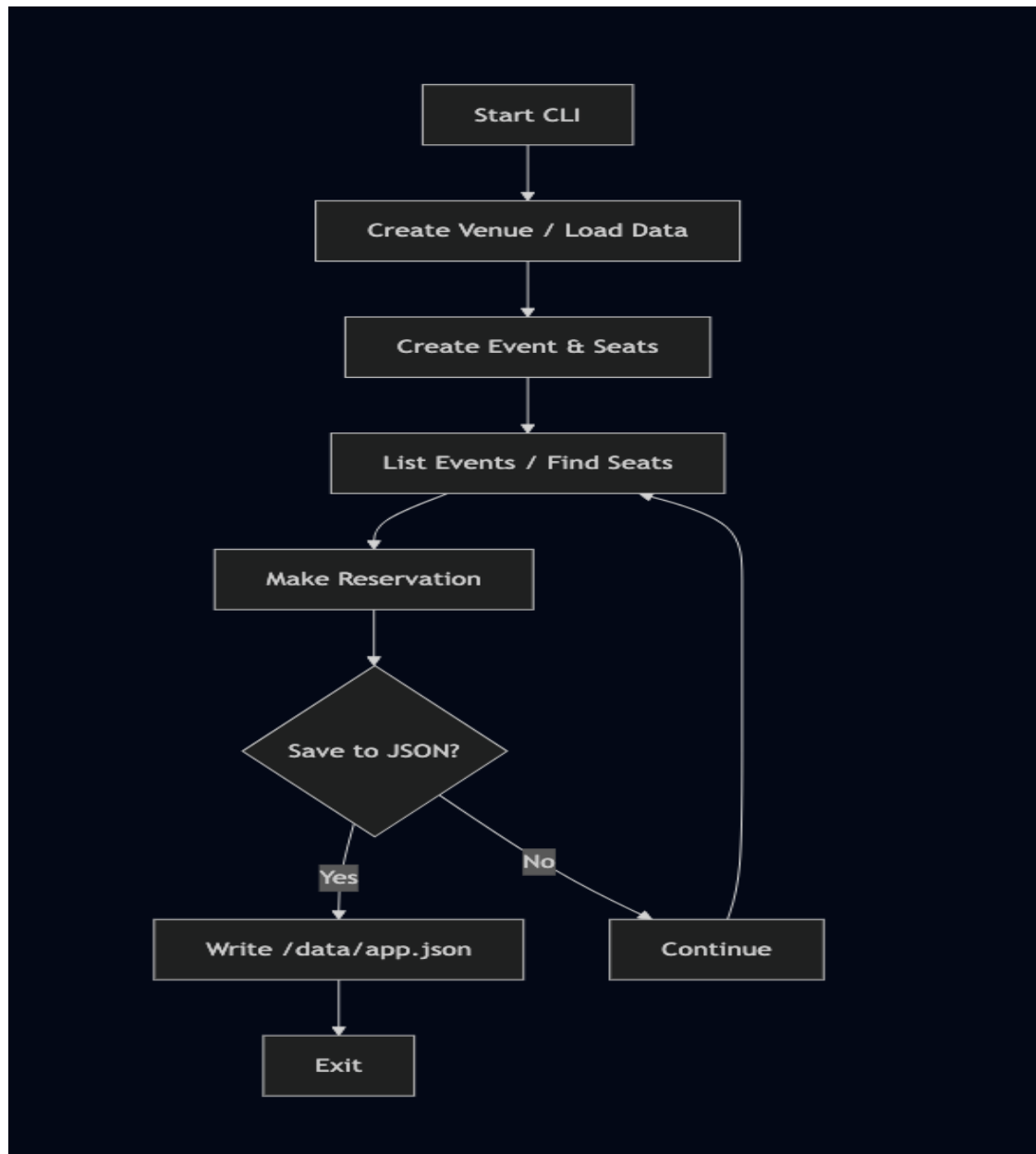
- **Core OOP & Design**
 - Classes vs interfaces; **encapsulation**; **immutability** (records/value objects); **composition over inheritance**; **cohesion & coupling**; **SOLID** (esp. SRP/ISP: keep classes small, interfaces focused).
 - **Enums with behavior** (state + methods) for discount types.
 - **Packages & visibility** (public/protected/package-private/private) to enforce module boundaries.
- **Core Collections & Generics**
 - Interfaces (List, Set, Map, Deque) vs implementations (ArrayList, LinkedList, HashSet, LinkedHashMap).
 - **Generics** variance basics; type inference; avoiding raw types.
 - **Equality contracts** (equals/hashCode) and their effect on HashMap/HashSet.
- **Streams & Functional Style**
 - map/filter/reduce, collect, Collectors.groupingBy/joining, method references; minimizing intermediate materialization; avoiding side-effects in streams.
- **Java Time (java.time)**
 - Instant, Duration, LocalDateTime, ZonedDateTime, ZoneId; time zone handling for events.
- **I/O & Serialization**
 - Files, Paths, **try-with-resources**; JSON save/load design (DTOs vs domain).
- **Error Handling & Testing**
 - **Checked vs unchecked** exceptions; defining **domain exceptions**; **fail fast** validation.

- **JUnit 5** structure; AAA pattern (Arrange-Act-Assert); assertion quality and deterministic tests.
- **Build & Run**
 - JDK 21 toolchain; Gradle/Maven basics; app layout and fat-jar creation.

Features (must-have)

- Manage venues/events/seats/reservations **in memory**.
- Save/load to **JSON**.
- Basic **reports** (seats per event, reservations per event).
- Friendly **errors; unit tests**.

Optional add-ons: Promo codes (enum), ticket export (text files).



Phase 1 — Core Java CLI foundations (OOP, Collections/Streams, Time, Testing).

Micro-prep

- CSV → objects → report via Streams
- Money value object (equals/hashCode/toString)
- Comparator<Seat> (section → row → number)
- Custom InvalidSeatException handled in main()

Testing — Step by Step

1. **Unit:** value equality; comparator order; validation throws; stream reports match fixtures.
2. **Persistence:** JSON save/load round-trip equals; corrupted file → friendly error.
3. **CLI flow:** create → reserve → save → reload; negative path (non-existent seat).

Links

- Collections: <https://docs.oracle.com/javase/tutorial/collections/>
- Streams: <https://www.oracle.com/technical-resources/articles/java/ma14-java-se-8-streams.html>
- dev.java Learn: <https://dev.java/learn/>
- Java Time: <https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html>
- Try-with-resources: <https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html>
- JUnit 5: <https://junit.org/junit5/docs/current/user-guide/>
- OOP refresher: <https://www.freecodecamp.org/news/object-oriented-programming-concepts-java/>

Phase 2 — Robust Java (Concurrency, Caching, Patterns)

Concepts You'll Learn (in depth)

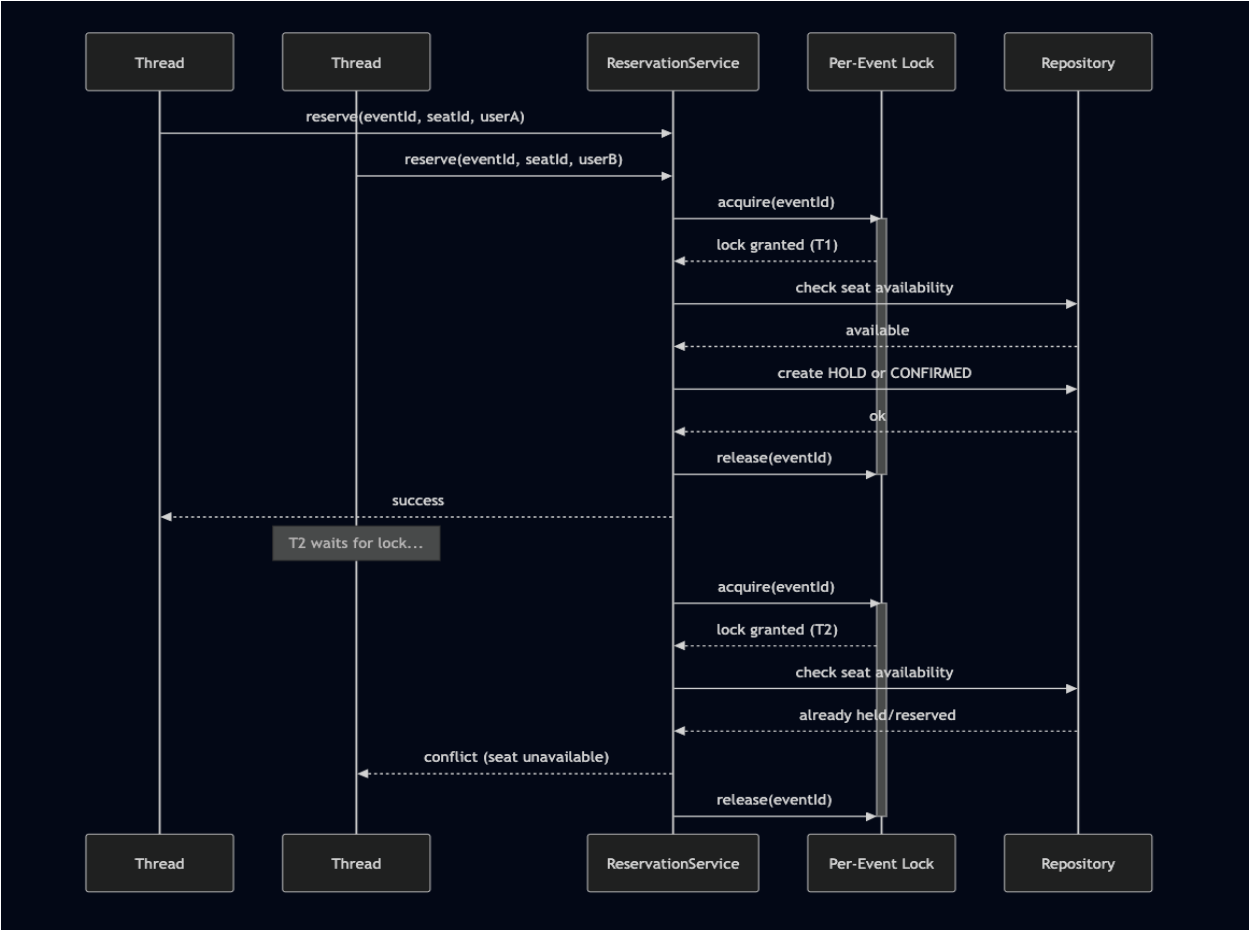
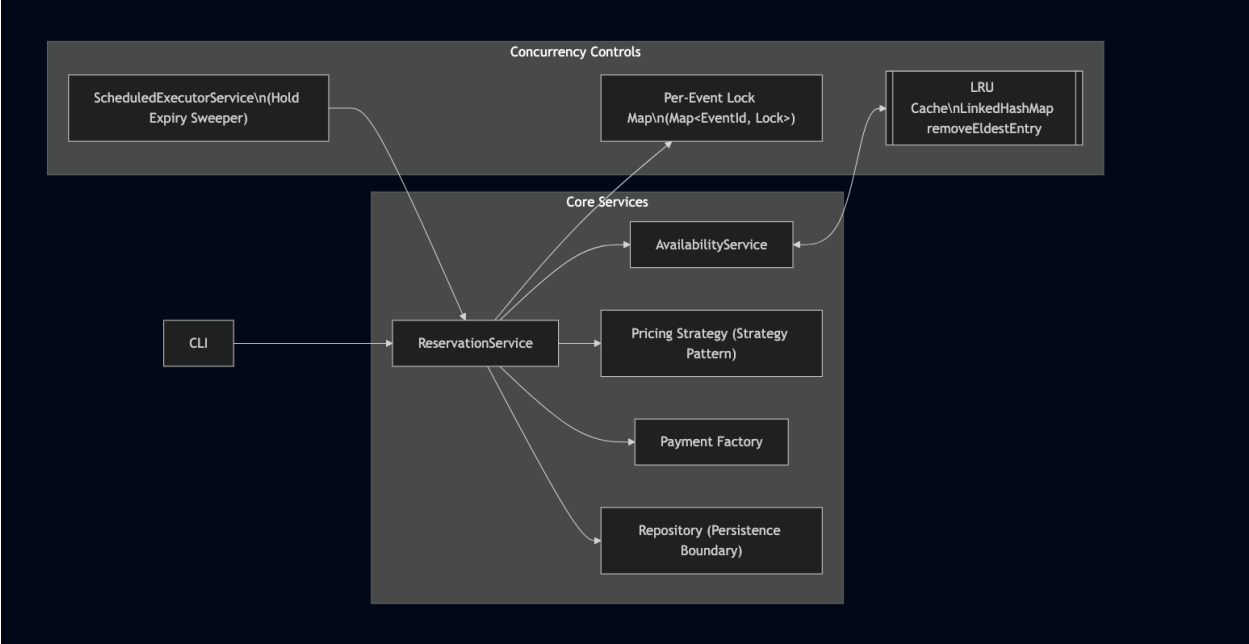
- **Concurrency Fundamentals**
 - **Threads vs thread pools;** when to use `ExecutorService` and `ScheduledExecutorService`.
 - **Java Memory Model basics:** visibility, **happens-before**, atomicity; why `volatile` isn't a lock.

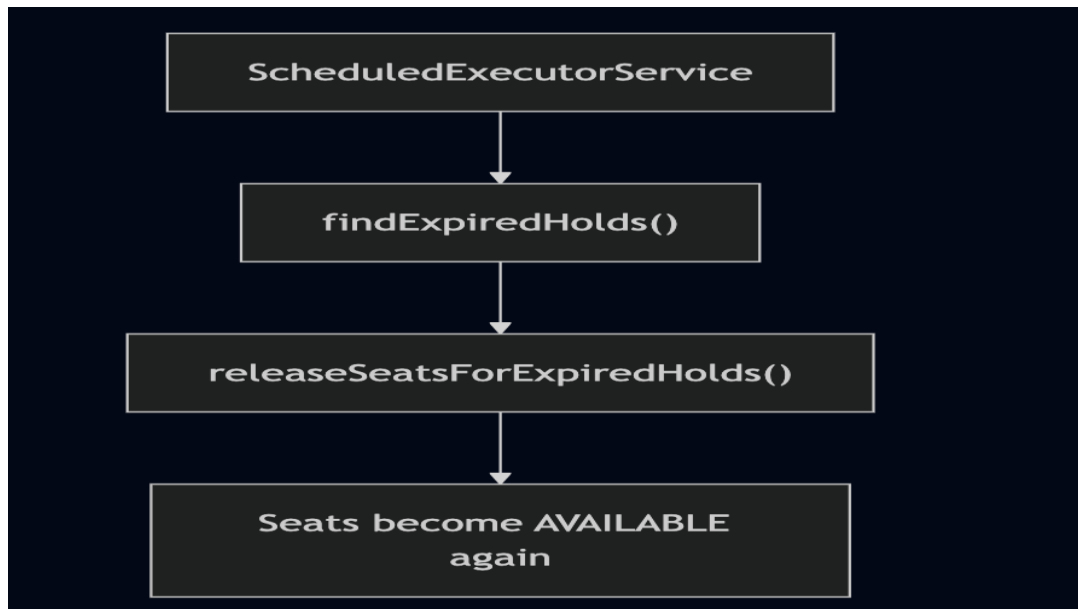
- **Locking strategies:** synchronized vs ReentrantLock vs StampedLock; **lock granularity** (per-event lock), **fairness**, **timeouts**.
- **Concurrent data structures:** ConcurrentHashMap, ConcurrentLinkedQueue; segmentation and contention.
- **Atomics:** AtomicInteger, AtomicReference for lock-free counters/flags.
- **Liveness:** deadlock (circular wait), starvation, priority inversion; **prevention** (lock ordering, timeouts, tryLock).
- **Idempotency & Retries**
 - Designing **idempotent** operations at boundaries; retry with **backoff**; idempotency keys (preview for Phase 4).
- **Caching Theory (pragmatic)**
 - **LRU** via LinkedHashMap#removeEldestEntry; TTL vs size-based eviction; **cache stampede** basics (single-flight, soft TTL).
- **Practical Patterns**
 - **Strategy** (pricing/seat selection), **Factory** (payment stub), **Repository** (persistence boundary), **Builder** (aggregate creation).
- **Diagnostics & Testability**
 - Concurrency test patterns (CountDownLatch, barriers); deterministic expiry; safe logging under concurrency.

Features (must-have)

- Holds with **expiry** (background sweeper).
- Per-event **locking** to prevent double-booking.
- **LRU cache** for hot lookups.
- Patterns wired into services.
- Concurrency tests & improved logs.

Optional add-ons: Token-bucket rate limiter; CSV import preview/validate.





Micro-prep

- Sweeper that purges expired holds
- ~30-line LRU with tests
- 10 threads race for same seat → fix via per-event lock

Testing — Step by Step

1. **Race test:** 10 threads → exactly one success, rest deterministic failures; repeat runs stable.
2. **Expiry:** set 2s TTL; assert expiresAt; second user blocked until sweep; after sweep seat available.
3. **LRU:** A,B,C → touch A → add D → B evicted; no ConcurrentModificationException under mild concurrency.
4. **Patterns:** swap strategies → different totals; factory selects provider; repo failure wrapped as domain error.

Links

- Concurrency: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>

- ExecutorService:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ExecutorService.html>
- Locks package:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/package-summary.html>
- Concurrent package:
<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html>
- LinkedHashMap LRU:
<https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html#removeEldestEntry-java.util.Map.Entry->
- Big-O cheat sheet (language-agnostic):
<https://wiki.python.org/moin/TimeComplexity>

Phase 3 — Spring Boot REST API + PostgreSQL

Concepts You'll Learn (in depth)

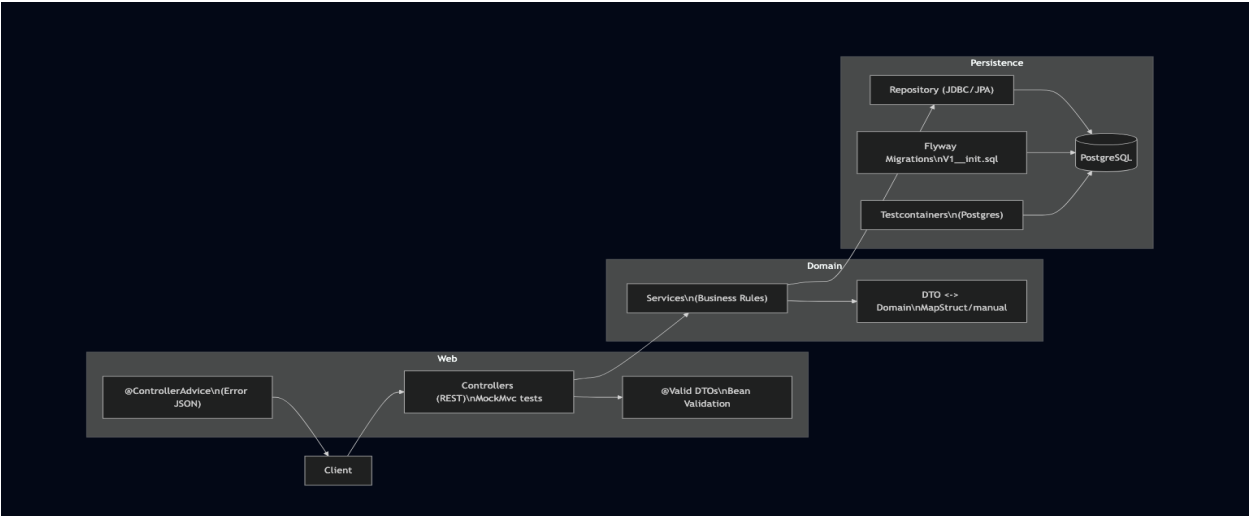
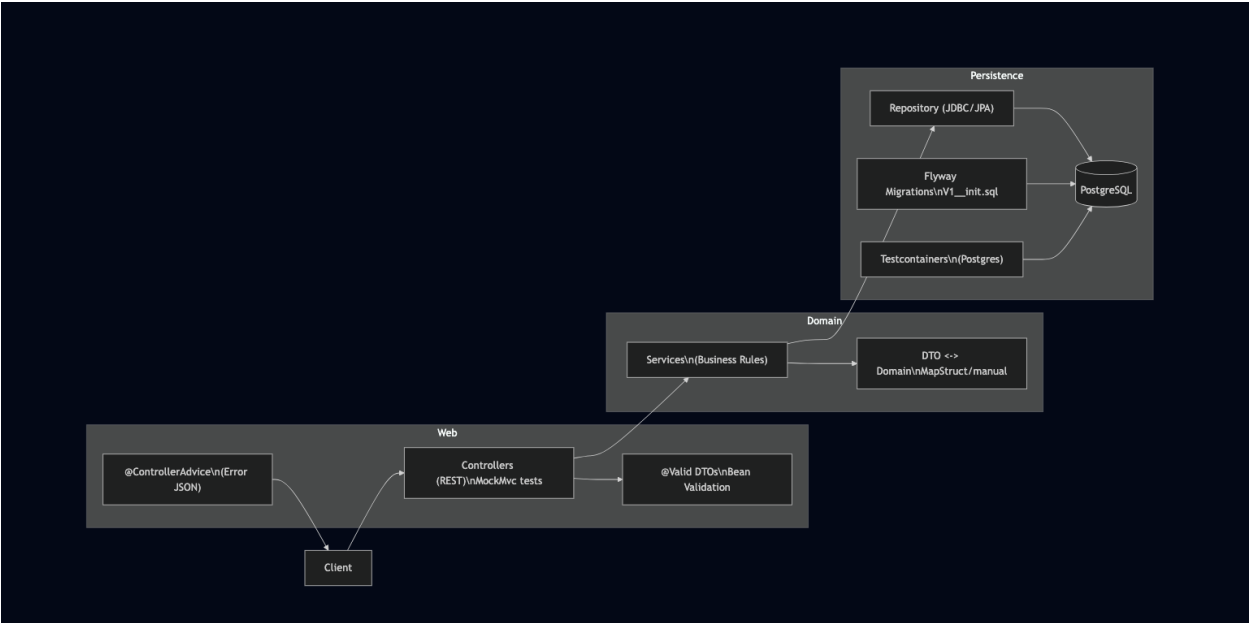
- **REST & HTTP Semantics**
 - Resource modeling; **idempotency** (GET/PUT/DELETE), **safety**, **201/204/409/422** usage; error contracts; pagination patterns (offset vs **cursor**).
- **Validation & DTO Design**
 - @Valid, constraint annotations, message templates; **DTO vs entity** separation; **Controller → Service → Repository** layering; **MapStruct** vs manual mapping.
- **Database Design**
 - **Normalization** (1NF/2NF/3NF) to avoid anomalies; **foreign keys**; **cascades**; **uniqueness** constraints like (event_id, seat_id) to prevent double booking.
- **Transactions & Isolation**

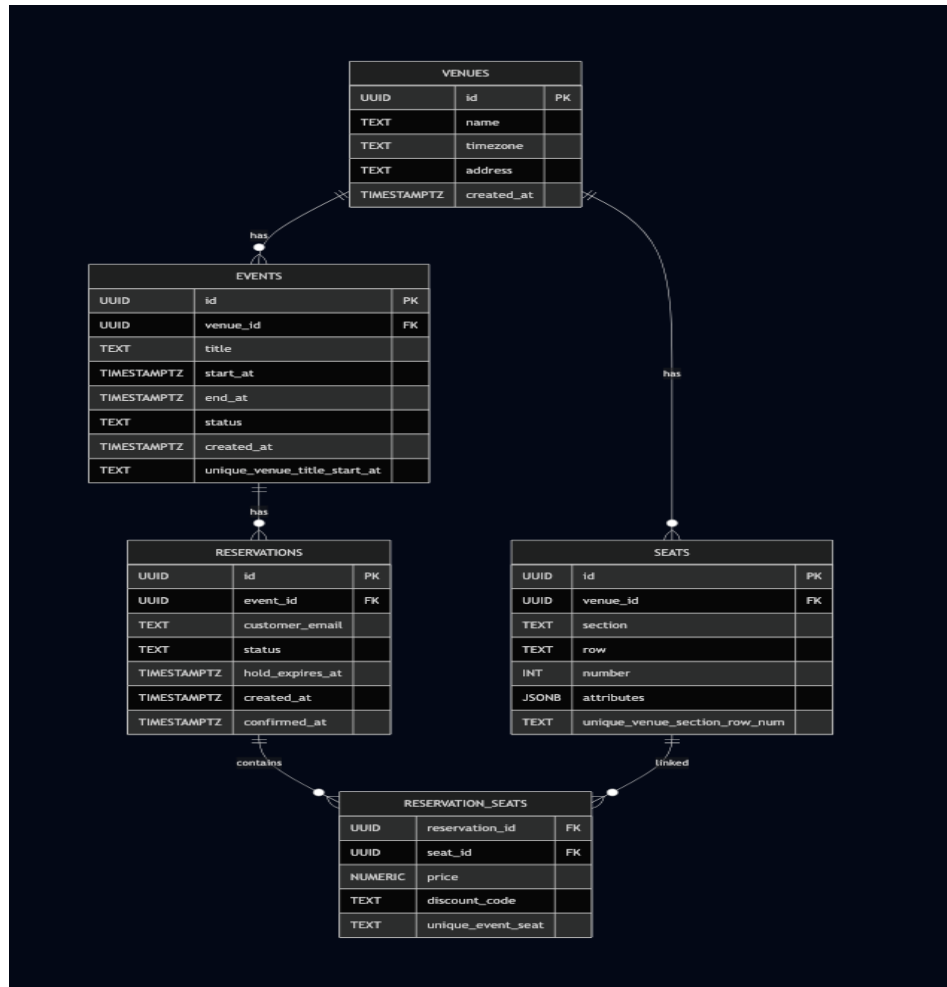
- **ACID**; isolation levels (**Read Committed, Repeatable Read, Serializable**) and anomalies (dirty/non-repeatable/phantom reads); choosing isolation for booking; **optimistic locking** with @Version vs constraint-based conflict handling.
- **Indexing & Performance**
 - Index selection; multi-column indexes; **EXPLAIN/EXPLAIN ANALYZE**; avoiding N+1 (even with JDBC/JPA).
- **Migrations & Testability**
 - **Flyway** versioned vs repeatable migrations; naming; repeatable idempotency; rollback strategy (forward-only).
 - **Testcontainers** for real Postgres in tests; **MockMvc** slice tests vs end-to-end tests.
- **Pooling & Ops**
 - **HikariCP** sizing (threads × CPU × DB limits); timeouts; leak detection; graceful shutdown.

Features (must-have)

- REST endpoints:
 - GET /api/events?from=&to=&q=&page=&size=
 - GET /api/events/{id}, GET /api/events/{id}/seats?onlyAvailable=true
 - POST /api/reservations/hold
 - POST /api/reservations/{id}/confirm (idempotent)
 - POST /api/reservations/{id}/cancel
- Global validation & error JSON via @ControllerAdvice.
- Flyway V1__init.sql with constraints/indexes.
- Transactions & uniqueness to prevent double-booking.
- Testcontainers/MockMvc tests.

Optional add-ons: Seat JSONB metadata; cursor pagination; /api/reports/events/{id}/summary.





Micro-prep

- Write V1__init.sql (venues, events, seats, reservations, reservation_seats)
- Implement findEventsBetween(from,to,search) (JDBC/JPA) + Testcontainers test
- Minimal GET /api/events with pagination + validation

Testing — Step by Step

1. **Flyway schema:** containers up, migrations run; verify tables/constraints/indexes via information_schema.
2. **Repo tests:** time-range search; availability excludes reserved; unique constraint triggers on double book (map to 409).
3. **Controller tests:** invalid page/size → 400; missing/invalid email → 400; unknown IDs → 404; repo exception → 500 with consistent schema.

4. **E2E**: create venue/event/seats → hold → confirm → availability reflects; confirm twice is idempotent; pagination correctness; JSONB round-trip (optional).

Links

- Spring REST: <https://spring.io/guides/gs/rest-service/>
- Spring JDBC: <https://spring.io/guides/gs/relational-data-access/>
- Spring JPA: <https://spring.io/guides/gs/accessing-data-jpa/>
- Bean Validation: <https://docs.spring.io/spring-framework/reference/core/validation/beanvalidation.html>
- Controller Advice: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-advice.html>
- PostgreSQL JSONB: <https://www.postgresql.org/docs/current/datatype-json.html>
- CREATE INDEX: <https://www.postgresql.org/docs/current/sql-createindex.html>
- Transaction Isolation: <https://www.postgresql.org/docs/current/transaction-iso.html>
- Flyway: <https://documentation.red-gate.com/fd/getting-started-with-flyway-184127223.html>
- Testcontainers: <https://testcontainers.com/guides/getting-started-with-testcontainers-for-java/>
- HikariCP: <https://github.com/brettwooldridge/HikariCP>

Phase 4 — Production-ish Spring (Security, Observability, Performance)

Concepts You'll Learn (in depth)

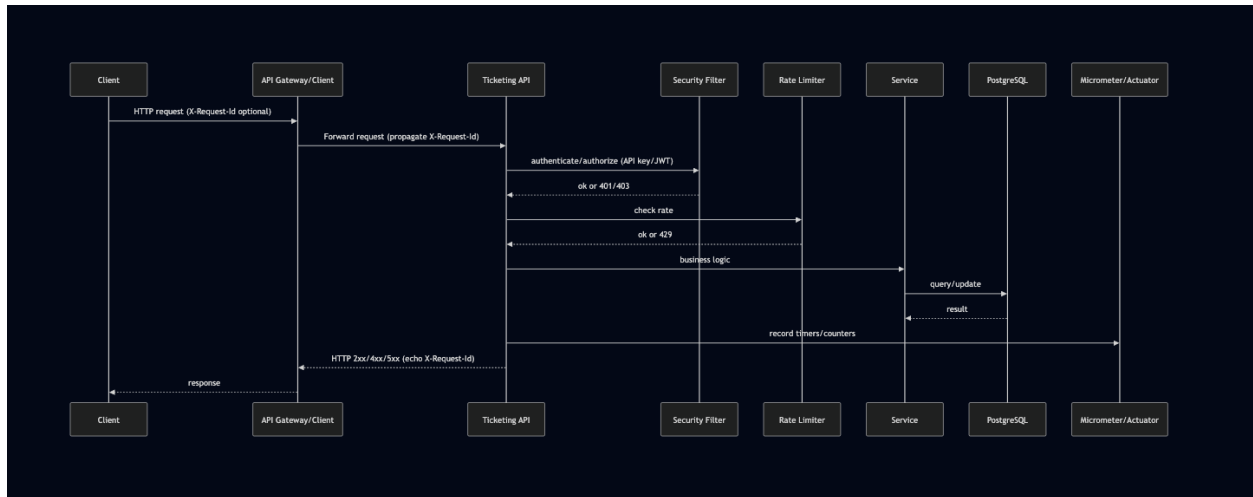
- **Security Foundations**
 - **API key vs JWT**: use-cases; stateless auth; signature/expiration/claims; header conventions; clock skew.
 - **Authorization** basics; protecting write endpoints; **CORS** preflight, allowed origins/headers/methods.

- **Threat modeling** lite: input validation, error leakage, brute-force, replay; **idempotency keys** for confirm/cancel.
- **Reliability & Resilience**
 - **Rate limiting** algorithms (fixed window, sliding window, token bucket/Leaky Bucket); choosing identifiers (IP/API key).
 - **Retries** with exponential backoff/jitter; **circuit breaker** (conceptual).
 - **Error budgets/SLOs**: availability & latency targets.
- **Observability (Three Pillars)**
 - **Metrics** (Micrometer counters/timers/gauges); RED/USE methods for services.
 - **Logs**: structured fields (traceId, userId, route, status, latency), log levels, rotation.
 - **Tracing** (concepts): correlation IDs (X-Request-Id) across hops.
- **Packaging & Config**
 - **Multi-stage Docker** builds; minimal runtime image; healthchecks.
 - **Externalized config** via env vars; **secrets** not baked into images.
- **Performance Engineering**
 - **Connection pool** tuning; EXPLAIN ANALYZE for hot queries; index coverage/selectivity.
 - **Load testing** methodology: define goals, warm-up, steady state; **p95/p99** interpretation; regressions.

Features (must-have)

- Security: API key or JWT for mutating endpoints; strict **CORS**.
- **Rate limiting** for write endpoints.
- **Actuator** /health, /metrics, /info; **Micrometer** metrics; **X-Request-Id** filter; structured logs.
- **Docker** multi-stage; **Compose** for app+db+healthchecks; env-driven config.
- **Performance**: Hikari tuning; **EXPLAIN ANALYZE**; **k6** smoke tests.

Optional add-ons: QR tickets; webhook on confirmation (retries/backoff); seat suggestion endpoint.



Micro-prep

- API key filter returning 401/403
- Enable Actuator; verify endpoints
- k6 script hammering POST /api/reservations/hold (20–50 VUs)

Testing — Step by Step

1. **Security:** unauthenticated → 401; wrong key → 403; correct → 2xx; CORS preflight as expected.
2. **Rate limiting:** 5 req/10s threshold; 6th → 429 + Retry-After; window reset works; counters per key/IP.
3. **Observability:** /actuator/health = UP; metrics include http.server.requests, hikaricp.*; logs contain X-Request-Id (provided or generated).
4. **Packaging:** Docker image builds/runs; Compose brings up app+db; healthchecks pass; env overrides applied.
5. **Performance:** seed 500 seats; 20–50 VUs for 1–3 min (70% GET seats, 20% hold, 10% confirm); assert http_req_failed < 1%, **p95** within target; DB integrity holds.

Links

- Actuator: <https://docs.spring.io/spring-boot/reference/actuator/index.html>
- Micrometer concepts: <https://micrometer.io/docs/concepts>
- Spring Security: <https://spring.io/guides/gs/securing-web/>

- k6 getting started: <https://grafana.com/docs/k6/latest/examples/get-started-with-k6/>
- Docker get started: <https://docs.docker.com/get-started/>
- Docker Compose quickstart: <https://docs.docker.com/compose/gettingstarted/>
- PostgreSQL EXPLAIN: <https://www.postgresql.org/docs/current/using-explain.html>

Study vs Skim

- **Study deeply:**
 - P1: Collections/Streams, Exceptions, Java Time, JUnit
 - P2: Java Memory Model basics; Locks & concurrent collections; Scheduling; LRU cache
 - P3: Transactions & isolation; Uniqueness constraints; Controllers/Validation; Flyway
 - P4: Auth & CORS; Rate limiting; Actuator/Micrometer; Docker multi-stage; Load test reading of p95
- **Skim, revisit when needed:**
 - Advanced JPA mappings; JWT internals; advanced k6/JMeter scripting; circuit breakers/tracing backends

Definition of Done (roll-up)

- **P1** — Domain corrects; JSON round-trip; CLI errors friendly; unit tests green.
- **P2** — Race test stable; expiry deterministic; LRU verified; patterns swappable.
- **P3** — Flyway schema valid; double-booking prevented; REST error/validation consistent; CI green.
- **P4** — Auth/CORS/rate-limit enforced; metrics & request IDs present; Docker/Compose healthy; smoke p95 within target; zero corruption.

